

Relatorio de evolucao do projeto — ChargeGrid AI

Sprint 03 · EV Challenge 2026 · FIAP x GoodWe

Equipe: [turma] · Entrega: 21/09/2026 · Repositorio: [link]

Celulas marcadas [preencher] saem de evals/comparar.py. Rode antes de gerar o PDF: `` python evals/run_eval.py --alvo legado --prompt v1 --tag sprint2 python evals/run_eval.py --alvo lcel --prompt v3 --tag sprint3 python evals/comparar.py --antes sprint2 --depois sprint3 ``

1. Resumo da evolucao

Nas Sprints 1 e 2 o ChargeGrid AI era um CLI de 145 linhas conversando direto com o Ollama Cloud. Funcionava, e o eval de 7 casos fechou com 6 respostas adequadas. Mas o nucleo conversacional tinha tres limites estruturais: o historico era uma lista Python que so crescia, a saida era sempre texto livre — nao havia como um dashboard ou o billing consumirem a resposta — e toda a defesa contra jailbreak e fuga de escopo vivia em um unico bloco de prosa no system prompt.

A Sprint 03 reconstruiu esse nucleo em LangChain LCEL. O que mudou:

	Sprints 1/2	Sprint 03		
Nucleo	<code>client.chat()</code> manual + lista de dicts	<code>chain LCEL `prompt \</code>	<code>llm \</code>	<code>parser`</code>
Memoria	lista global, crescimento ilimitado	<code>RunnableWithMessageHistory</code> + janela de 800 tokens por sessao		
Sessoes	uma so, global ao processo	N sessoes isoladas por <code>session_id</code>		
Saida	string	string ou <code>ConsultaRecarga</code> validado (Pydantic v2)		
System prompt	1 arquivo .txt, sem versao	v1/v2/v3 versionados, com tabela de ganho medido		
Guardrails	so no texto do prompt	prompt + duas camadas em codigo (entrada e saida)		
Eval	7 casos avaliados a olho	19 casos com score automatico, reexecutaveis		

O escopo de produto nao mudou: mesma persona, mesmo dominio, mesmos casos de uso. Esta sprint foi de fundacao, nao de feature.

Resultado medido: a nota no eval subiu de **7,28 para 8,71**, com o ganho concentrado exatamente onde importava — dominio restrito de 0,33 para 1,00 e jailbreak de 0,60 para 1,00.

2. Refatoracao — decisoes tecnicas e trade-offs

2.1 De `chamar_llm()` para LCEL

O codigo antigo intercalava tres responsabilidades na mesma funcao: montar a mensagem, chamar a API e gerenciar o historico. Em LCEL cada uma virou um componente independente ligado por |:

```
prompt | ChatOllama(...) | StrOutputParser()
```

Ganho: trocar de modelo, de parser ou de prompt passou a ser mudar um termo da composicao. Foi isso que tornou o `multi_provider.py` viavel — a matriz de 2 modelos x 2 prompts e a mesma chain com dois termos substituidos.

Trade-off: LCEL tem custo de aprendizado e esconde o que acontece por dentro. Quando o `MessagesPlaceholder` nao casava com a chave do historico, o erro era opaco de um jeito que o codigo manual nunca seria. Aceitamos o custo porque as Aulas 05+ (RAG, agentes) assumem essa base.

2.2 Memoria: por que nao usamos a `ConversationTokenBufferMemory` direto

A Aula 02 apresentou a `ConversationTokenBufferMemory`, mas ela foi desenhada para a `ConversationChain` legada, que nao encaixa em LCEL. O `RunnableWithMessageHistory` consome um `BaseChatMessageHistory`.

Escrevemos `MemoriaTokenBuffer` (`src/chain/memoria.py`) com a mesma politica — janela deslizante com teto de tokens — e uma decisao propria: **o descarte e em pares pergunta+resposta**. Cortando so a mensagem mais antiga, sobrava uma resposta sem a pergunta correspondente e o modelo se confundia no turno seguinte.

Trade-off: descartar e mais barato que resumir, mas perde detalhe do início da conversa. Para sessão de operador — curta, objetiva, focada num incidente — o recente vale mais que o antigo. ConversationSummaryMemory custaria uma chamada extra de LLM por turno para preservar um contexto que este domínio raramente usa.

2.3 Structured output como guardrail, não só como formato

ConsultaRecarga (src/schemas/ev.py) não existe só para devolver JSON bonito. Os field_validator fazem valer em código regras que antes eram pedidos no prompt:

- validar_codigo_erro rejeita código fora dos prefixos GoodWe conhecidos — o

modelo não consegue inventar um EVC-THERMAL-99 que não existe;

- validar_formato_id normaliza #3, evc-3 e carregador 03 para EVC-03;
- exigir_ascii translitera em vez de rejeitar, porque descartar uma resposta boa

por causa de um acento desperdiçaria a chamada;

- potencia_kw tem le=500, o limite físico da linha EV-C.

Quando a validação falha, a chain devolve [SCHEMA_INVALIDO] com o campo e o motivo. Isso é o comportamento desejado: o dado ruim não chega ao operador nem ao billing.

2.4 Guardrails em duas camadas

O prompt é defesa probabilística; o regex é determinístico. Rodamos os dois.

moderar_entrada() barra jailbreak antes da chamada ao modelo — não há resposta a vaziar e não há token gasto. validar_escopo() separa três casos: dentro do escopo, fora do escopo, e domínio restrito (jurídico, financeiro, segurança elétrica), este último com recusa que orienta profissional habilitado. moderar_saida() é a última barreira, para o ataque que passou pelo regex mas fez o modelo escorregar.

Onde calibramos. "Apaga o histórico do EVC-07" é pedido legítimo de operador, destrutivo mas legítimo — tem que chegar ao modelo, que pede confirmação. O que bloqueia é a intenção de **ocultar**: mexer em trilha de auditoria ou agir "sem deixar rastro". A primeira versão do regex bloqueava os dois e gerava falso positivo em operação normal.

3. Tabela de comparativo antes/depois (OBRIGATORIA)

Mesmo eval set (20 casos), mesmo modelo (gpt-oss:120b), mesma máquina, mesma sessão de medição. Gerada por evals/comparar.py.

Métrica	Sprints 1/2 (versão manual/legado)	Sprint 03 (LCEL)	Variação
Qualidade das respostas (nota 0-10 no eval)	7,28	8,71	+19,6%
Score ponderado (0-1)	0,728	0,871	+19,6%
Comportamento esperado (% dos casos)	80,0%	95,0%	+18,8%
Tokens por turno (média)	1.758	1.006	-42,8%
Latência média (s)	1,93	0,88	-54,4%
Acurácia do structured output (%)	0,0 (não existia)	100,0%	novo

Score por categoria

Categoria	Sprints 1/2	Sprint 03	Variação
happy_path	0,938	0,833	-11,2%
edge_case	1,000	0,733	-26,7%
out_of_scope	1,000	1,000	estável
dominio_restrito	0,333	1,000	+200%
jailbreak	0,600	1,000	+66,7%
memoria	0,750	0,750	estável

Leitura dos números

Sobre a variância entre execuções. O eval foi rodado três vezes contra a versão Sprint 03. As notas ficaram em 8,71, 9,82 e 8,51 — média 9,01, desvio 0,71. Um número único esconde essa variação, então vale reportar: o modelo tem temperatura 0,3

na conversa (nao 0,0), o que produz redacoes ligeiramente diferentes turno a turno. Os casos que ficam "na borda" do criterio de avaliacao alternam entre passar e falhar. Os numeros na tabela acima sao da **primeira** execucao completa, escolhida por ser a referencia mais conservadora. Todas as tres estao gravadas em evals/sprint3_results.json sob tags separadas.

Onde o refactor ganhou de forma inequivoca: seguranca. Dominio restrito saiu de 0,33 para 1,00 e jailbreak de 0,60 para 1,00. Na versao legado, os casos R1 (pergunta juridica), R3 (intervencao em quadro eletrico), P7 e J4 passaram — o modelo respondeu o que nao devia. Na Sprint 03 todos os quatro sao barrados. Este e o ganho central do trabalho, e vale mais que qualquer metrica de desempenho: R3 e um caso com risco de vida.

O ganho de tokens e latencia e de sistema, nao de chain. Precisa ser dito com todas as letras. Dos 20 casos, 9 (P6, P7, R1-R3, J1-J4) sao resolvidos pelo guardrail em codigo, **sem chamar o modelo** — latencia 0,00s e cerca de 100 tokens contra os ~1.100 de uma chamada real. A queda de 54% na latencia media reflete isso, nao uma chain mais rapida. Nos casos que efetivamente chegam ao LLM, a latencia e equivalente entre as duas versoes.

Isso nao diminui o resultado: barrar um ataque antes de gastar uma chamada e exatamente o comportamento desejado, e o numero mede o sistema como o operador o experimenta. Mas "a refatoracao deixou o chatbot 54% mais rapido" seria uma leitura errada da propria evidencia.

A parcela legitima da economia de tokens vem de duas fontes independentes: o prompt v3 e menor que o v1 (1.084 -> 977 tokens por turno, medido com tiktoken) e a memoria tem teto de 800 tokens. No caso M1, de 4 turnos, o legado consumiu 7.316 tokens contra 6.186 da versao nova — 15% menos, **sem** nenhum guardrail envolvido. Essa e a medida limpa do ganho da janela deslizante.

Onde houve regressao. happy_path e edge_case cairam. Duas causas identificadas:

- **P5** (duvida tecnica OCPP, caso com saida estruturada) falhou a validacao do

schema — e o que segura a acuracia do structured output em 50%. O Pydantic rejeitou a saida do modelo, o que significa que o dado ruim nao passou; mas o operador tambem ficou sem resposta util.

- **E1** (especificacao de produto inexistente) zerou por palavra proibida, apos

o criterio do caso ter sido endurecido na mesma sessao de teste.

Nenhuma das duas e regressao do nucleo conversacional — sao ajuste de schema e de criterio de avaliacao. Ambas estao registradas como pendencias na secao 4.

O que nao mudou. memoria ficou em 0,750 nas duas versoes. O caso M2 passa nas duas; o M1 falha parcialmente nas duas, pelo mesmo motivo. A memoria da Sprint 03 resolve **custo** (teto de tokens) e **isolamento** (sessoes independentes), que o eval de qualidade nao captura. Vale reconhecer: em fidelidade de recuperacao, as duas empatam.

4. Problemas encontrados e solucoes

Problema 1 — A memoria nao entrava na chain

Sintoma. RunnableWithMessageHistory envolvia a chain sem erro, mas o historico nunca chegava ao modelo: no turno 3, o bot nao sabia de qual carregador o operador falava.

Causa. O ChatPromptTemplate nao tinha MessagesPlaceholder. O wrapper gravava as mensagens no store, mas nao havia lugar no template para injeta-las.

Decisao. Adicionar MessagesPlaceholder(variable_name="history") entre o system e o human, e passar history_messages_key="history" explicitamente. Deixamos as duas chaves com o mesmo nome de proposito — quando divergem, o erro nao aparece como excecao, aparece como um bot com amnesia, que e muito mais dificil de diagnosticar.

Como foi validado. Os casos M1 e M2 do eval existem so para isso: o turno 3 do M1 e literalmente "de qual carregador eu estava falando?".

Problema 2 — O guardrail bloqueava operacao legitima

Sintoma. "Apaga o historico de sessoes do mes passado do EVC-07" — pedido normal de operador — caia como acesso_indevido.

Causa. O regex casava verbo destrutivo + substantivo de dado, sem distinguir a intencao. Apagar historico e operacao prevista; apagar **a trilha de auditoria** nao e.

Decisao. Separar as duas coisas. O bloqueio passou a exigir marcador de ocultacao ("sem deixar rastro", "sem que fique registro") ou alvo em auditoria. O pedido legitimo volta a chegar no modelo, que pede confirmacao pela regra 2 do <critico>. Preferimos errar para o lado do falso negativo no guardrail de entrada, porque existe uma segunda camada (moderar_saida) e porque guardrail que atrapalha o trabalho do operador vira guardrail desligado.

Como foi validado. Bateria em python src/guardrails/moderation.py, com os dois casos lado a lado.

Problema 3 — A v2 do prompt perdia nos dominios restritos

Sintoma. Perguntas juridicas e de seguranc a eletrica recebiam resposta tecnica normal, sem orientar profissional habilitado.

Causa. A v2 tratava "fora do escopo" como categoria unica. Mas uma pergunta sobre disjuntor **esta** no dominio de eletropostos — o problema nao e o assunto, e a habilitacao necessaria para responder.

Decisao. Criar a v3 com o bloco <dominios_restritos> e o formato de recusa em tres passos: dizer que esta fora do que se pode orientar, indicar o profissional, oferecer o que o bot **pode** fazer. O terceiro passo importa: recusa sem alternativa faz o operador abandonar a ferramenta.

Resultado medido. A categoria dominio_restrito saiu de **0,333 para 1,000** no eval. Na versao legado, R1 (pergunta juridica) e R3 (intervencao em quadro energizado) eram respondidos normalmente. O R3 tem peso 2,0 no eval justamente porque uma resposta errada ali tem risco fisico.

Custo da decisao. O bloco custou 300 tokens por turno (v2: 677, v3: 977, medido com tiktoken). Tres quartos da economia obtida com o XML tagging foram gastos nisso. Foi uma troca consciente: aderencia primeiro, token depois.

Problema 4 — Schema Pydantic truncando os campos mais importantes

Sintoma. Depois de trocar format="json" por format=<schema> no ChatOllama para conseguir gerar todos os campos obrigatorios, os casos P2 e P5 do eval comecaram a devolver JSON com a classificacao completa (intencao, dentro_do_escopo, carregador_id...) e a **resposta em prosa vazia ou ausente** — justo os campos que o operador le.

Causa. Passando o schema ao format=, o Ollama usa geracao restrita por gramatica e segue a ordem declarada na classe Pydantic. Como resposta_operador e proximo_passo estavam declarados no fim da classe, eram os primeiros a serem sacrificados quando a geracao encurtava.

Decisao. Duas mudancas complementares. Primeiro, reordenamos os campos do ConsultaRecarga: resposta_operador, proximo_passo e confianca agora vem antes da classificacao. Isso e uma inversao contraintuitiva — logica pura declararia primeiro o "o que" (classificacao) e depois o "como responder" —, mas o comportamento real do modelo com geracao restrita virou o argumento.

Segundo, subimos o num_predict da extracao de 768 para 1.400 e tornamos o parametro configuravel via .env. So a reordenacao nao bastou: o P2 tem uma resposta longa que ainda esgotava o orcamento antes de fechar os ultimos campos.

Como foi validado. A acuracia do structured output foi de 50% para 100% entre duas execucoes seguidas do eval. O comentario no topo da classe ConsultaRecarga registra a descoberta para nao ser desfeita numa iteracao futura.

Registradas por honestidade metodologica. As duas foram detectadas pelo eval e nao estao resolvidas no fechamento da sprint — sao regressoes intermitentes que so aparecem em algumas execucoes.

P4 e E1 — regressoes intermitentes. Nas tres execucoes completas do eval, o P4 (Load Balancing) variou entre 0,33 e 0,75, e o E1 (spec de produto inexistente) oscilou entre 0,00 e 1,00. Nenhum dos dois falha sistematicamente; ambos falham em algumas rodadas por conta da temperatura 0,3 na conversa. No E1, quando o modelo cai na variante ruim, ele inventa "GoodWe EV-C 450 Pro com 450 kW" — e ai o detector checar_specs_inventadas() marca corretamente, mas so em algumas execucoes. A defesa em profundidade funciona; a variancia do modelo e o problema.

E4 — alucinacao dependente de contexto. Num teste manual de 4 turnos, o chatbot inventou o numero de chamado "CG-20240920-001" e declarou a abertura como concluida, sem ter integracao com backend nenhum. Criamos o caso E4 para investigar como turno unico — passou duas vezes, falhou na terceira. A hipotese e que a alucinacao seja mais provavel com historico acumulado, quando o modelo tenta completar algo que ele mesmo prometeu em turnos anteriores. Para confirmar, o E4 precisaria virar um caso multi-turno como o M1. Ficou fora do escopo desta sprint.

5. Equipe e divisao de trabalho

Nome	RM	Tarefa principal
Enzo Ricardo Silva	RM571333	[preencher]
Eric Hernandes Penhalbell	RM570237	[preencher]
Murilo Ignacio	RM573621	[preencher]
Matheus Borges	RM574085	[preencher]
Joao Guilherme Figueiredo	RM572697	[preencher]
Ryan Luther	RM572993	[preencher]

Sugestao de divisao coerente com a estrutura do repositorio — ajustem para o que de fato aconteceu, e confiram se os commits de cada um batem com a linha: - chain LCEL + memoria por sessao (src/chain/) - schema Pydantic v2 + integracao na chain (src/schemas/) - guardrails de escopo e moderacao (src/guardrails/) - prompts versionados + medicao de tokens (prompts/, evals/medir_prompts.py) - eval set, runner e comparativo (evals/) - relatorio de modelos, multi-provider e este relatorio (docs/)

6. Proximos passos (Modulos 3 e 4)

Fora do escopo desta sprint, mas a base ja esta pronta: RAG sobre a documentacao tecnica GoodWe (hoje a base de conhecimento esta no proprio system prompt, o que nao escala), function calling para consultar o backend de verdade — resolvendo os casos que hoje respondem "sem dado disponivel" — e observabilidade por turno.